手続き型プログラミング発展 レポート (大課題 1)

24B30992 樋口陽希

1　課題についての説明

大容量のテキストデータを効率よく分析するツールを作成する．入力するコマンドにもよるが，1MB につき，0.8 秒程度の処理時間で分析することに成功した．

テキストデータの内容は日本語と数字は存在しないものとし，アルファベットのみを対象とする．また，単語ごとの区切りは，空白，カンマ，ピリオド，改行を認めており，大文字と小文字の区別はしない．単語同士が 2 文字以上の区切り文字で区切られている場合にも対応している．

出力はすべて小文字で行う．

2　プログラムの説明

2.1　コマンドとプログラムの全体的な流れ

プログラムは以下のコマンドを受け付ける．

1. -s　テキストファイルの統計情報を表示する．
2. -f (prefix)　接頭辞が (prefix) である単語を表示する．
3. -t (n) 出現頻度 top n の単語を表示する．
4. -w (word) 単語 (word) が存在するか否かを表示する．

各コマンドの詳細は後述する．

次に，プログラムの全体的な流れを説明する．

どのコマンドが入力されても，最初にハッシュテーブルというシステムを構築することで，処理速度を向上させている．

ハッシュテーブルのシステムを説明する．

ハッシュテーブルは，単語ごとにハッシュ関数を用いてハッシュ値を計算し，そのハッシュ値をキーとして，キーに対応する配列のインデックスのノードに単語の出現回数，単語の長さ，単語の出現位置等，必要な情報を記録するというシステムである．

ハッシュノードに格納されている情報は以下の通り．

1. 単語自身
2. 単語の出現回数
3. 単語の長さ

4. 単語の出現位置
5. 単語が最長かどうかのフラグ
6. 単語が最短かどうかのフラグ
7. 次のノードへのポインタ

ハッシュ値の計算手法は以下の通り．

まず，最初のハッシュ値を 5381 とする．この数字は，結果的にあらゆる文字列に対して散らばったハッシュ値を生成することができると経験的に判明しているマジックナンバーである．

次に，ハッシュ値を 33 倍したのちに，文字の ASCII コードを加えて，それを新たなハッシュ値とするという操作を単語のすべての文字に対して繰り返す．33 倍というのも，散らばったハッシュ値を生成することができると経験的に判明しているマジックナンバーである．

すべての文字を処理した後，ハッシュ値をハッシュテーブルの配列のサイズで割った余りをとることで，ハッシュ値をハッシュテーブルの配列のインデックスに変換できる．

ハッシュ値は unsigned long 型であり，長い単語でハッシュ値を計算しようとすると，オーバーフローが発生する．しかし，このオーバーフローは意図的に発生させており，オーバーフローが発生した場合のハッシュ値は，ハッシュ値を unsigned long 型の最大値で割った余りとなるため，問題なく処理できる．

これにより，ハッシュ値はアナグラムとなっている単語に対しても，異なるハッシュ値を返し，さらに，あらゆる単語に対して適度に散らばりのあるハッシュ値を生成することができる．

さらに，すべてのハッシュノードには次のノードへのポインタを持たせている．また，ハッシュテーブルの配列のサイズは固定長で十分である．

もし，異なる単語同士でハッシュ値が同じになってしまう（衝突してしまう）場合は，そのハッシュ値の最後尾のノードの次のポインタに新たなノードを追加することで対応している．

これを繰り返すことにより，大きなテキストファイルであっても固定長の配列でハッシュテーブルを構築することができ，また，単語の探索も O(1) の時間で行うことができる．

このハッシュテーブルを構築したのち，コマンドごとの処理を行う．

コマンドを一つ実行するだけであれば，ハッシュテーブルを構築してもしなくても，処理にかかる時間はほとんど変わらない．

しかし，複数のコマンドを同時に処理する場合は，ハッシュテーブルを構築した方が圧倒的に速く処理を行うことができる．

このプログラムの使い道は，word 等のテキストファイルエディタにおいて，リアルタイムで情報を更新，表示することにあると考えている．

よって，複数のコマンドの同時処理に強い，ハッシュテーブルを構築することとした．

2.2 コマンド 1 -s

コマンド -s は，テキストファイルの統計情報を表示する．

具体的には，

1. アルファベットおよび区切り記号の総数
2. 単語の総数
3. ユニークな単語の総数

4. 単語の平均長
5. 最も長い単語とその長さ
6. 最も短い単語とその長さ

を表示する．これらすべての情報は，すべてのハッシュノードを参照することで簡単に入手できる．

2.3 コマンド 2 -f (prefix)

コマンド -f (prefix) は，接頭辞が (prefix) である単語を表示する．

一つの接頭辞だけを検索するのであれば，ハッシュノードをすべて参照してもかまわないが，複数の接頭辞を検索しても現実的な時間で処理が行える，トライ木というシステムを構築した．

トライ木は，各文字が一つのノードに対応し，すべてのノードが次のアルファベットに対応する 26 個の子ノードと，単語の末尾か否かを示すフラグ，さらにそれぞれの単語がテキストファイルのどこに存在するかの情報を持つ．

単語の終了フラグが立っていないノードの単語の存在位置は-1 としている．

まず，あらゆる単語の開始を宣言する根ノードを作成する．

そのあと，ハッシュテーブルのすべての単語を参照し，単語を追加する．

単語の追加は，根ノードからスタートし，単語の各文字を順に参照しながらノードをたどり，その文字に対応する子ノードが存在しなければ，その文字に対応する子ノードを作成する．

そして，単語の末尾に到達したら，そのノードの末尾フラグを立てる．

トライ木を構築し終わった後の接頭辞の検索の方法は以下の通り．

まず，指定された接頭辞に対応するノードをトライ木からたどる．

もし，接頭辞に対応するノードが存在しなければ，接頭辞に対応する単語は存在しないと判断し，その旨を表示する．

もし，接頭辞に対応するノードが存在すれば，接頭辞に対応するノードの末尾のノードを基準とし，そのノードにつながるすべてのノードを深さ優先探索で探索する．探索の際，末尾のフラグが立っているノードに到達したら，そのノードに対応する単語を表示する．

2.4 コマンド 3 -t (n)

コマンド -t (n) は，出現頻度 top n の単語を表示する．n は整数を指定する．

これも，ハッシュテーブルのすべてのすべての単語を参照してもよいが，より効率的な方法として，ヒープを用いた．

ヒープは要素数 n の完全二分木（ただし，葉ノードの階層はすべて埋まっているとは限らない）であり，親ノードの単語の出現回数が子ノードの単語の出現回数よりも小さいという性質を常に保つ．ヒープは完全二分木であるため，ハッシュテーブルのようにノードで管理する必要はなく，ハッシュノードのポインタを格納する配列で管理することができる．

以下にヒープの構築方法を説明する．

ヒープの要素がすべて埋まるまでは，ヒープの性質に従ってハッシュノードのポインタを配列に追加していく．

ヒープの要素がすべて埋まった後は，根ノードの単語の出現回数と追加するか検証したいハッシュノードの

単語の出現回数を比較し，もし，検証中のハッシュノードの単語の出現回数が根ノードの単語の出現回数よりも大きい場合は，根ノードを交換し，ヒープの性質を保つようにヒープを再構築する．

以上の操作をすべてのハッシュノードに対して行う．

2.5 コマンド 4 -w (word)

コマンド -w (word) は，単語 (word) が存在するか否かを表示する．これは，ハッシュテーブルを構築した後，存在確認したいハッシュ値を計算し，そのハッシュ値に対応するハッシュノードを参照することで，O(1) の時間で確認できる．もし，ハッシュノードが存在しなければ，単語は存在しないと判断して出力する．ハッシュテーブルの強力さが最もわかりやすく示されるコマンドである．

3 プログラムのコンパイルの仕方

プログラムは以下のコマンドでコンパイルした．

```
gcc -O2 -g -lm -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wwrite-strings -Wredundant-decls -Wstrict
```

このコマンドは TOJ 上のコンパイルコマンドとほぼ一致しているが，-fsanitize=address -fsanitize=undefined については専用のコンパイル用のライブラリを用意する必要があるため，省略している．また，デバッグを簡単に行うために，-O0 を指定している．

4 プログラムの実行方法と想定した出力例

プログラムは以下のコマンドで実行した．

```
./2qfinal.exe

./2qfinal.exe 2qfinal.txt

./2qfinal.exe non_existent.txt

./2qfinal.exe large_random_article.txt -s

./2qfinal.exe large_random_article.txt -s -t 5

./2qfinal.exe large_random_article.txt -f pre

./2qfinal.exe large_random_article.txt -f ""

./2qfinal.exe large_random_article.txt -f

./2qfinal.exe large_random_article.txt -t 10
```

```
./2qfinal.exe large_random_article.txt -t five

./2qfinal.exe large_random_article.txt -t

./2qfinal.exe large_random_article.txt -t 0

./2qfinal.exe large_random_article.txt -t -5

./2qfinal.exe large_random_article.txt -w hello

./2qfinal.exe large_random_article.txt -w ""

./2qfinal.exe large_random_article.txt -w

./2qfinal.exe large_random_article.txt -s -t 5 -w the

./2qfinal.exe large_random_article.txt -w a -f un -t 10 -f pre -w on

./2qfinal.exe large_random_article.txt -w a -t 10 -f prescri -s

./2qfinal.exe large_random_article.txt -x
```

想定した出力例は以下のとおりである.
正しいコマンド

```
./2qfinal.exe large_random_article.txt -w a -t 10 -f prescri -s
```

が入力された場合；

```
s_tf: 1
t_tf: 1
n = 10
f_tf: 1
prefix = prescri
w_tf: 1
word = a
=== TEXT STATISTICS ===
Total characters: 15767280
Total words: 1456541
Unique words: 233899
```

```
Average word length: 9.55
Longest words:
"tetraiodophenolphthalein"
"scientificophilosophical"
"formaldehydesulphoxylate"
"thyroparathyroidectomize"
"pathologicopsychological"
(24 characters)
Shortest words:
"a"
"b"
"c"
"d"
"e"
"f"
"g"
"h"
"i"
"j"
"k"
"l"
"m"
"n"
"o"
"p"
"q"
"r"
"s"
"t"
"u"
"v"
"w"
"x"
"y"
"z"
(1 character)
=== WORDS STARTING WITH "prescri" ===
Found 11 matches:
-"prescribable" (position 4338213, 4957474, 13924821)
-"prescribe" (position 7540007, 8671541, 9184801, 11595512 ...)
```

```
-"prescriber" (position 2757316, 11553452, 12461186, 14682740)
-"prescriptibility" (position 3682983, 4185018, 4340233, 7099135 ...)
-"prescriptible" (position 1057409, 2659507, 9728345, 9846842 ...)
-"prescription" (position 1670020, 11654614, 12337747, 14584042)
-"prescriptionist" (position 6068115, 7170241, 11678738, 14255043)
-"prescriptively" (position 5788655, 6464328, 8021927, 8887752 ...)
-"prescriptiveness" (position 11139250, 13683266, 15181110)
-"prescriptorial" (position 1784076, 1834166, 2034335, 4400068 ...)
-"prescrive" (position 1389427, 2965397, 6693024, 6975675 ...)
=== TOP 10 WORDS ===
1. "may" - 28 occurrences
2. "chromatoscopy" - 13 occurrences
3. "miniconjou" - 11 occurrences
4. "conormal" - 11 occurrences
5. "pterygostaphyline" - 9 occurrences
6. "playlet" - 7 occurrences
7. "mechanicality" - 7 occurrences
8. "spaller" - 7 occurrences
9. "phytogenetic" - 6 occurrences
10. "polymastigate" - 6 occurrences
=== WORD SEARCH ===
Word "a" found: 19 occurrences (positions 654291, 1022988, 1508218, 3484986 ...)
```

不正なコマンド

```
./2qfinal.exe large_random_article.txt -x
```

が入力された場合；

```
Invalid option
./textproc <input_file> [options]
Options:
  -s             Show statistics
  -f <prefix>    Find words starting with prefix
  -t <n>         Show top n most frequent words
  -w <word>      Search for specific word
```

また，動的なメモリ確保の途中でメモリ確保に失敗した場合もそれに対応する出力を行う．

テスト用のテキストファイルは 15MB 程度の大きさであり，ランダムな英単語がカンマ，空白，ピリオド，改行で区切られるように Python で生成したものである．

このテキストファイルに対するハッシュテーブルを生成するコマンドの実行時間は約 12 秒である．

各コマンドで，単語の出現回数が 1 であった場合，三項演算子を用いて”occurrences”の出力を”occurrence”としたり，単語の長さが 1 であった場合，”characters”の出力を”character”としたりするなど，単数形，複数形の細かい出力の調整を行っている．

5　プログラムの制限または想定される問題点

このプログラムは，日本語や数字を含むテキストデータには対応しておらず，日本語や数字は区切り文字として認識されるという制限がある．また，アルファベット以外の文字はすべて区切り文字として認識されるため，英単語の短縮形や略語などに使われるアポストロフィは正しく認識されない．また，255 文字以上の長さの単語は正しく認識されない．さらに，ハッシュテーブルの配列のサイズは固定長であり，そのサイズは数 MB のテキストファイルの単語数を十分にカバーできるように設定しているが，数 GB レベルの大きさのテキストファイルに対しては，ハッシュテーブルの強みが発揮されなくなる可能性がある．

6　授業の感想

C 言語の仕組みについてはよく理解できたが，B 課題，C 課題で作らなければならないプログラムが複雑で難解であったため大変だった．

とある回の B 課題で出てきた Btree の実装は，本来は学士 3 年で学ぶ内容であるとの話を聞いて驚いた．